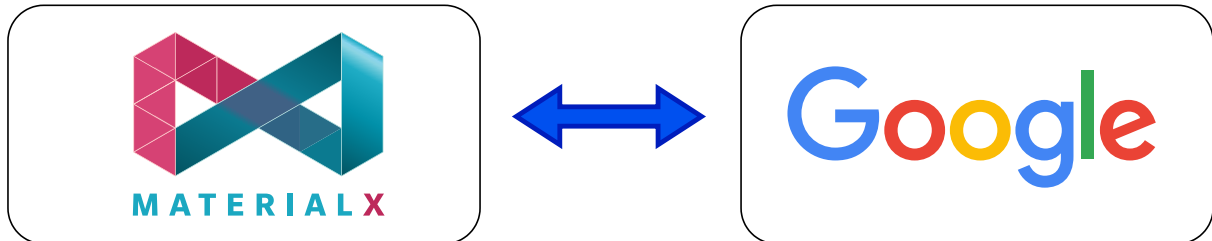


MaterialX Protobuf Converter



Python utilities for converting MaterialX documents to and from Protocol Buffer format.



What is Protocol Buffers?

[Protocol Buffers \(protobuf\)](#) is Google's mechanism for serializing structured data.

Benefits of Protocol Buffer Format

Protobuf focuses on data integrity, version safety, and cross-language consistency. Key benefits include:

- **Compact:** Binary format is smaller than XML
- **Fast:** Efficient serialization and deserialization
- **Cross-platform:** Language-agnostic format works across different platforms and programming languages
- **Typed:** Strong typing for data integrity
- **Versioning:** Built-in support for schema versioning and backward compatibility

Overview

This project provides tools to:

- Convert MaterialX XML documents to Protocol Buffer format
- Serialize MaterialX documents to binary `.mxpb` files
- Export MaterialX documents to JSON format
- Convert Protocol Buffer documents back to MaterialX XML
- Generate visual diagrams of MaterialX document structure

Prerequisites

macOS Setup

Install Protocol Buffers compiler:

```
brew install protobuf
```

Windows Setup

Install Protocol Buffers compiler:

```
choco install protoc
```

Linux Setup

Install Protocol Buffers compiler:

```
sudo apt-get install -y protobuf-compiler
```

Python Setup

1. Install Python protobuf library:

```
pip install --only-binary=all protobuf
```

2. Install MaterialX Python library:

```
pip install MaterialX
```

Python Tests

Run tests with:

```
pytest tests/ [--verbose]
```

If you run with verbose output all tests should pass::

```
tests/test_materialx.py::test_file_exists[standard_surface_chess_set.json] PASSED [ 3%]
tests/test_materialx.py::test_file_exists[standard_surface_chess_set.mtlx] PASSED [ 7%]
tests/test_materialx.py::test_file_exists[standard_surface_chess_set.mxb] PASSED [ 10%]
tests/test_materialx.py::test_file_exists[standard_surface_chess_set_converted.mtlx] PASSED [ 14%]
tests/test_materialx.py::test_file_exists[standard_surface_chess_set_from_pb.mtlx] PASSED [ 17%]
```

```
tests/test_materialx.py::test_file_exists[unlit_cross.json] PASSED
[ 21%]
...
tests/test_materialx.py::test_mxp_b_to_mtlx[standard_surface_chess_set.mxp_b] PASSED [ 96%]
tests/test_materialx.py::test_mxp_b_to_mtlx[unlit_cross.mxp_b] PASSED [100%]

===== 28 passed in 0.22s
=====
```

C++ Setup

For the C++ version, you'll need:

1. MaterialX C++ Library (version 1.39.4 or later)

- Download from: <https://github.com/AcademySoftwareFoundation/MaterialX>
- Build and install following their instructions

2. Protocol Buffers C++ Library

- **Windows:** `choco install protobuf` or use `vcpkg`
- **macOS:** `brew install protobuf`
- **Linux:** `sudo apt-get install libprotobuf-dev protobuf-compiler`

3. CMake (version 3.14 or later)

See [cpp/README.md](#) for detailed C++ build instructions.

Protocol Buffer Schema

The schema defines two main message types:

- **Attribute:** String key value pair. This is added to allow usage of repeated attributes as opposed to usage of maps. Maps in ProtoBuf are ordered which is undesirable for MaterialX where attribute order matters for types such as node inputs and outputs as these define the signature of the node.
- **MaterialXElement:** Represents a MaterialX element with name, type, attributes, and child elements.
- **MaterialXDocument:** Represents the root of the element hierarchy and contains top-level attributes.

This schema reflects the simplicity of the MaterialX representation.

A schema **Version** message is also provided for version management including allowing for an upgrade mechanism if ever required.

The schema is defined in the `materialx.proto` file.

Generating Code from Proto Schema

Python Bindings

Using the `materialx.proto`, Python bindings can be generated with the following command:

```
protoc --python_out=. materialx.proto
```

This generates `materialx_pb2.py` which contains the Protocol Buffer message classes.

C++ Bindings

For C++ development, generate C++ protobuf code:

```
protoc --cpp_out=./cpp materialx.proto
```

This generates `materialx.pb.h` and `materialx.pb.cc` in the `cpp` directory.

Automated Generation

Use the provided script to generate both Python and C++ code:

```
./generate_stubs.sh
```

File Structure

Python Version

```
source/
├── materialx.proto           # Protocol Buffer schema definition
├── materialx_pb2.py         # Generated Python protobuf bindings
├── materialx_serializer.py  # Conversion logic between MaterialX and
Protobuf
├── materialx_serializer.pyi # Type hints for Python
├── main.py                  # Command-line utility for conversion
└── data/
    └── *.mtlx               # Sample MaterialX documents
```

C++ Version

```
cpp/
├── CMakeLists.txt          # CMake build configuration
```

```
|— build.sh / build.bat      # Build scripts
|— main.cpp                  # Command-line utility (C++)
|— materialx_serializer.h    # Serializer class declarations
|— materialx_serializer.cpp  # Serializer class implementations
|— README.md                 # C++ specific documentation
```

Usage

The library provides both Python and C++ implementations with identical functionality.

Python Version

MaterialX to Protobuf Conversion

Convert a MaterialX document to Protobuf format:

```
python main.py path/to/<document>.mtlx
```

Or using the installed command:

```
materialxpb path/to/<document>.mtlx
```

This command can optionally generate four output files:

1. **.json** - JSON representation
2. **.mxpb** - Binary Protocol Buffer format (compact serialization)
3. **.mmd** - Mermaid diagram
4. **_converted.mtlx** - Round-trip conversion back to MaterialX XML (for validation)

Example

```
python main.py standard_surface_chess_set.mtlx --json --write_mermaid
```

Outputs:

- **standard_surface_chess_set.json** - JSON format
- **standard_surface_chess_set.mxpb** - Binary protobuf
- **standard_surface_chess_set_diagram.mmd** - Mermaid diagram

Protobuf to MaterialX Conversion

Convert a Protobuf binary file back to MaterialX XML:

```
python main.py path/to/<document>.mxpb
```

This will generate a MaterialX XML file named `<document>_from_pb.mtlx`.

C++ Version

The C++ version provides the same functionality with identical command-line arguments:

Building the C++ Version

```
cd cpp
./build.sh          # On Unix-like systems
# or
build.bat           # On Windows
```

See `cpp/README.md` for detailed build instructions.

Using the C++ Executable

```
./materialxpb document.mtlx
./materialxpb document.mtlx --json --write_mermaid
./materialxpb document.mtxpb
```

All command-line options are identical between Python and C++ versions.

Command-Line Options

Both Python and C++ versions support these options:

Option	Short	Description
<code>--json</code>	<code>-j</code>	Output JSON representation of the Protobuf document
<code>--convert_back</code>	<code>-cb</code>	Convert back to MaterialX after Protobuf conversion
<code>--write_mermaid</code>	<code>-wm</code>	Output Mermaid diagram of the Protobuf document
<code>--output_folder</code>	<code>-of</code>	Output folder for converted files
<code>--debug_pb_doc</code>	<code>-dbg</code>	Enable debug output (1=simple, 2=compact, 3=full)

Core Components

The library is implemented in both Python and C++ with equivalent functionality.

Python Implementation (`materialx_serializer.py`)

MaterialXToProtobuf

Converts MaterialX documents to Protocol Buffer format:

- Extracts document-level attributes
- Traverses the MaterialX document
 - Extracts element names, types, and attributes
 - Recursively converts child elements

ProtobufToMaterialX

Converts Protocol Buffer format back to MaterialX:

- Creates MaterialX document
- Reconstructs MaterialX document attributes
- Restores element hierarchy and attributes

Util

Utility functions for debugging and visualization:

- `from_string()` - PB doc from string
- `to_string()` - PB doc to string
- `to_json()` - PB doc to json
- `generate_mermaid_diagram()` - Generate Mermaid diagrams for visualization

C++ Implementation (cpp/)

The C++ version provides the same functionality with native performance:

- **materialx_serializer.h/cpp**: Core conversion classes
 - `MaterialXToProtobuf` - Converts MaterialX to Protobuf
 - `ProtobufToMaterialX` - Converts Protobuf to MaterialX
 - `Util` - Serialization, JSON export, and debug tools
 - `VersionUpgrader` - Schema version management
- **main.cpp**: Command-line interface matching the Python version

See `cpp/README.md` for C++-specific details.

Sample Files

The repository includes sample MaterialX documents:

- `standard_surface_chess_set.mtlx` - Chess set with standard surface shader
- `unlit_cross.mtlx` - Simple unlit cross shader

Each sample includes its converted outputs as appropriate (`.json`, `.mxpb`, `_converted.mtlx`, `from_pb.mtlx`).

Visualization

Enable Mermaid diagram generation in `main.py` by setting `write_mermaid = True`. This generates a `.md` file with a Mermaid diagram showing the document structure.